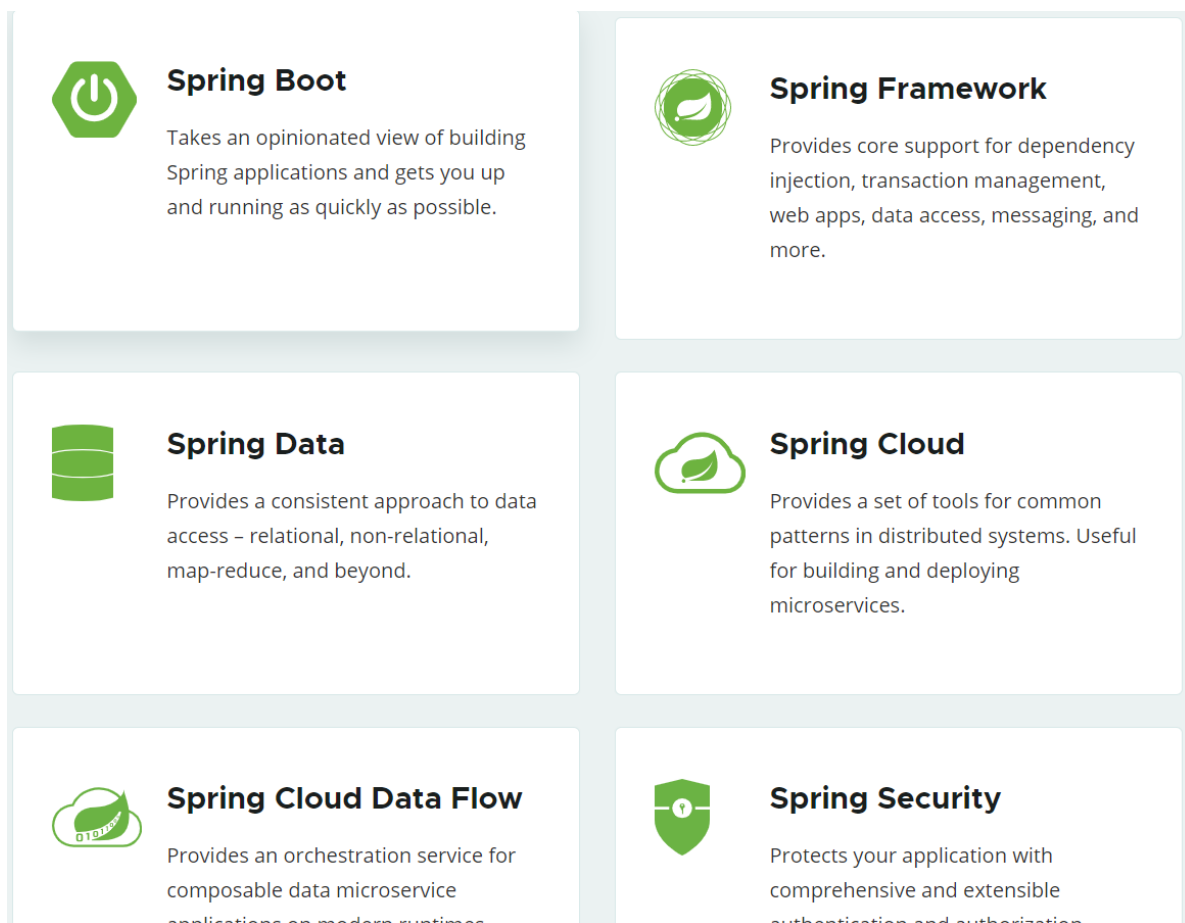


1 认识Springboot

1.1 什么是Springboot

Springboot是Spring项目中的一个子工程，与我们所熟知的Spring-framework 同属于spring的产品：

官网地址：<https://spring.io/projects>



我们可以看到下面的一段介绍：

Takes an opinionated view of building production-ready Spring applications. Spring Boot favors convention over configuration and is designed to get you up and running as quickly as possible.

翻译一下：

用一些固定的方式来构建生产级别的spring应用。Spring Boot 推崇约定大于配置的方式以便于你能够尽可能快速的启动并运行程序。

其实人们把Spring Boot 称为搭建程序的 脚手架。其最主要作用就是帮我们快速的构建庞大的spring项目，并且尽可能的减少一切xml配置，做到开箱即用，迅速上手，让我们关注与业务而非配置。

1.2 为什么要学习Springboot

java一直被人诟病的一点就是臃肿、麻烦。当我们还在辛苦的搭建项目时，可能Python程序员已经把功能写好了。为了解决这些问题，然后出现了一个框架叫做Spring，Spring翻译过来就是春天，意味着程序员的春天来了，但是真的是这样吗？其实大佬们创建出Spring这个框架本质上就是为了给程序员们减负，为了使我们构建项目更加便捷，为了让我们开发更加快速，但是呢，早期的Spring框架还是会有很多问题，最主要的两点就是：

- 复杂的配置

项目各种配置其实是开发时的损耗，因为在思考 Spring 特性配置和解决业务问题之间需要进行思维切换，所以写配置挤占了写应用程序逻辑的时间。

- 一个是混乱的依赖管理

项目的依赖管理也是件吃力不讨好的事情。决定项目里要用哪些库就已经够让人头痛的了，你还要知道这些库的哪个版本和其他库不会有冲突，这难题实在太棘手。并且，依赖管理也是一种损耗，添加依赖不是写应用程序代码。一旦选错了依赖的版本，随之而来的不兼容问题毫无疑问会是生产力杀手。

而SpringBoot让这一切成为过去！

Spring Boot 简化了基于Spring的应用开发，只需要“run”就能创建一个独立的、生产级别的Spring应用。Spring Boot为Spring平台及第三方库提供开箱即用的设置（提供默认设置，存放默认配置的包就是启动器），这样我们就可以简单的开始。多数Spring Boot应用只需要很少的Spring配置。

我们可以使用SpringBoot创建java应用，并使用java -jar 启动它，就能得到一个生产级别的web工程。

1.3 Springboot的特性

Spring Boot 主要目标是：

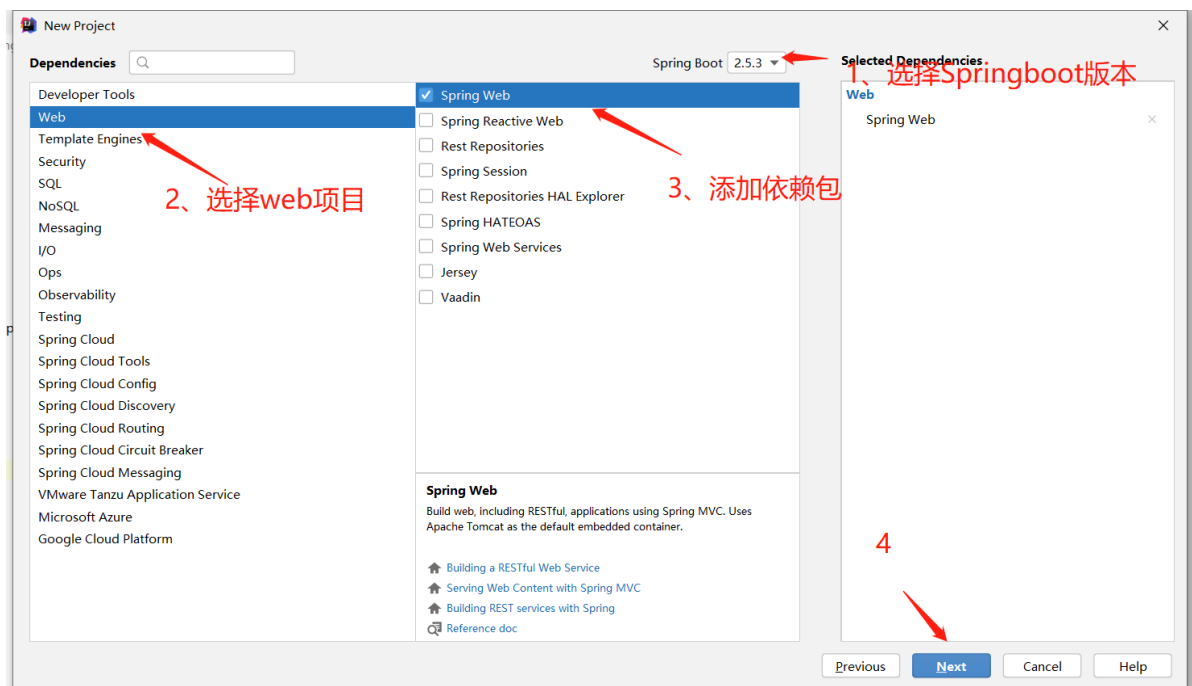
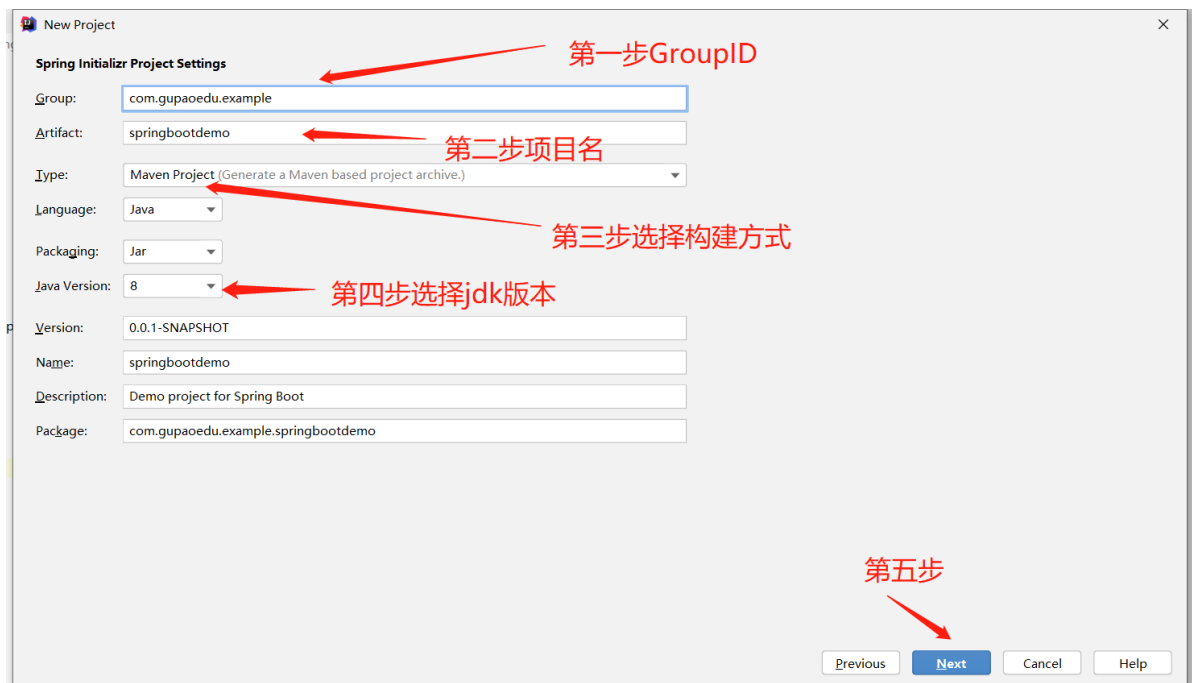
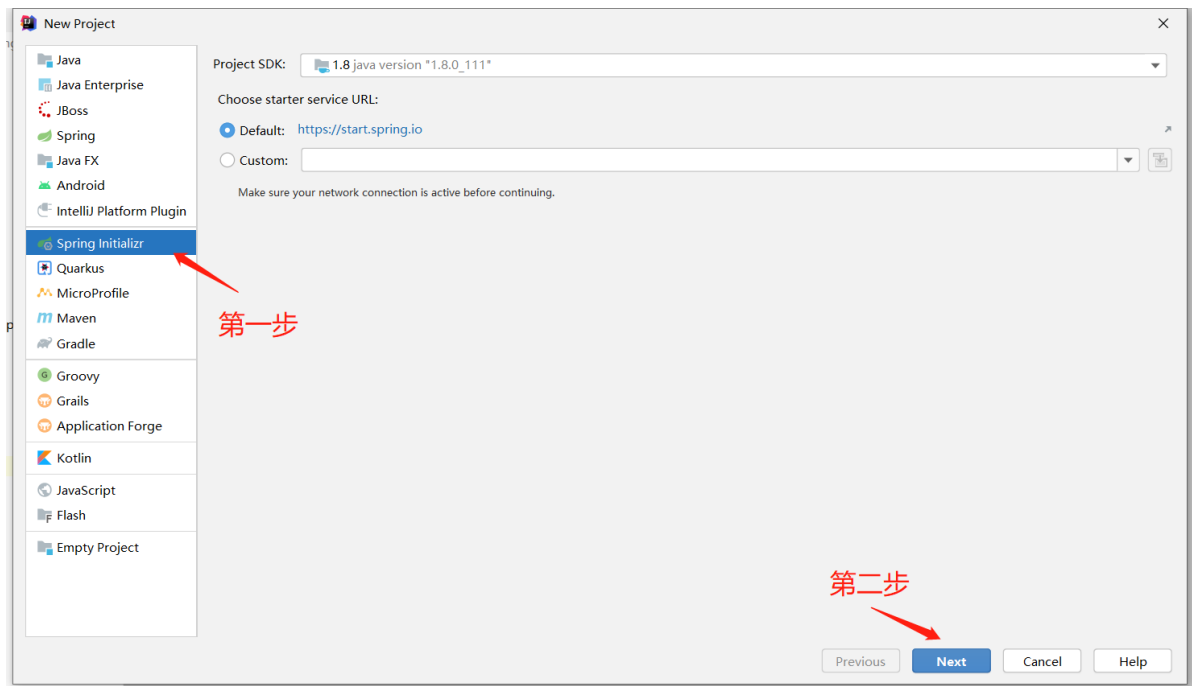
- 为所有 Spring 的开发者提供一个非常快速的、广泛接受的入门体验
- 开箱即用（启动器starter-其实就是SpringBoot提供的一个jar包），但通过自己设置参数（.properties），即可快速摆脱这种方式。
- 提供了一些大型项目中常见的非功能性特性，如内嵌服务器、安全、指标、健康检测、外部化配置等
- 绝对没有代码生成，也无需 XML 配置。

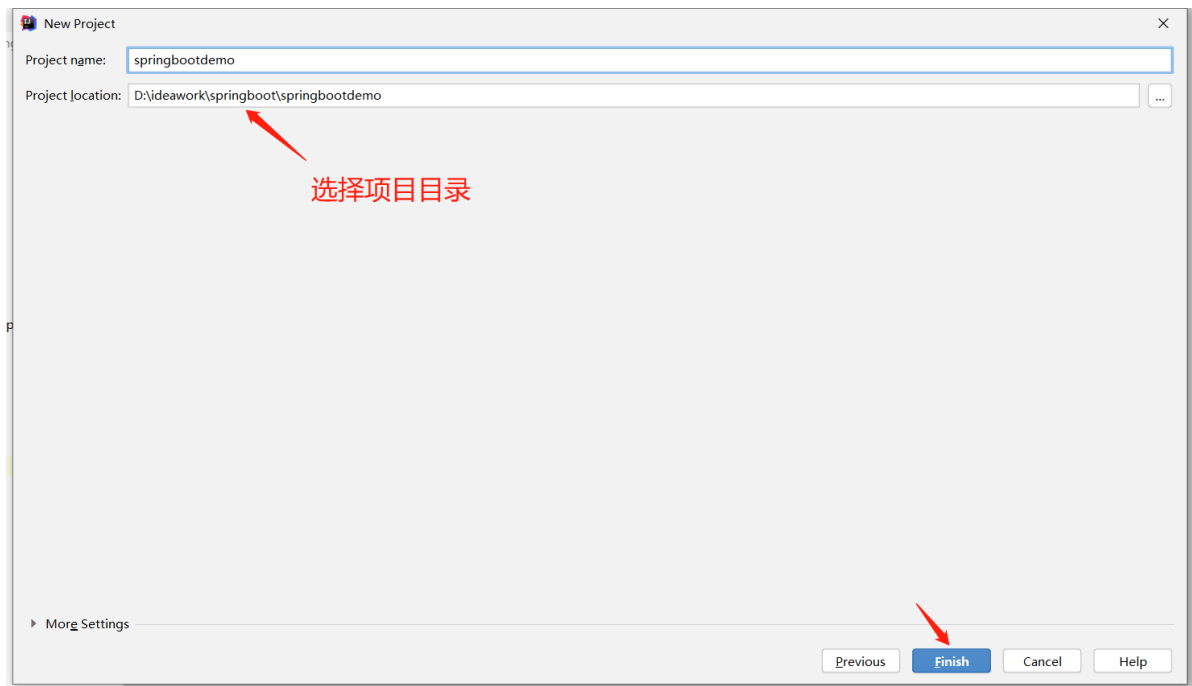
自动配置、起步依赖(Starter)、Actuator、命令行界面(CLI) 是 Spring Boot 最重要的 4 大核心特性，其中 CLI 是 Spring Boot 的可选特性，虽然它功能强大，但也引入了一套不太常规的开发模型，因而这里仅关注其它 3 种特性。

2 Springboot快速入门

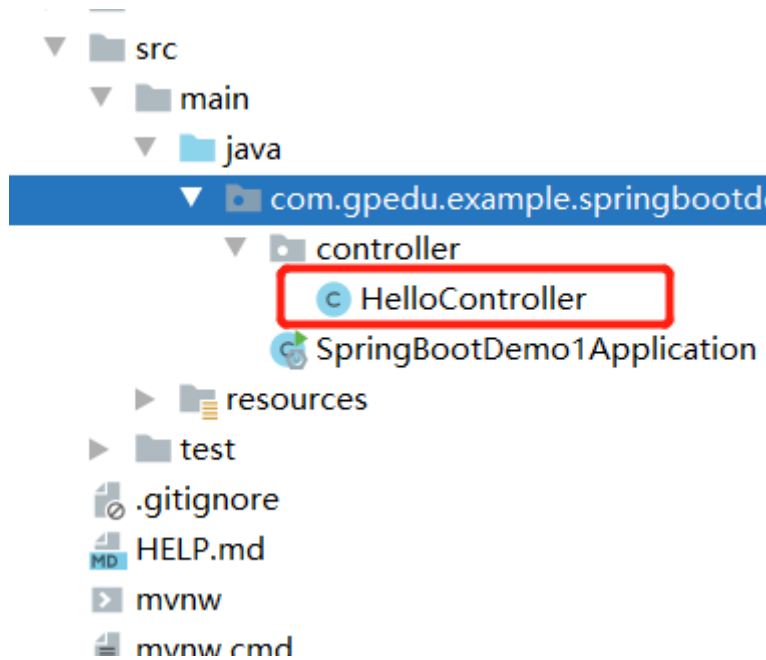
接下来，我们就来利用Springboot快速搭建一个web工程

2.1 创建工程





2.2编写controller



代码：

```
@RestController
public class HelloController {
    @GetMapping("hello")
    public String hello(){
        return "hello, spring boot!";
    }
}
```

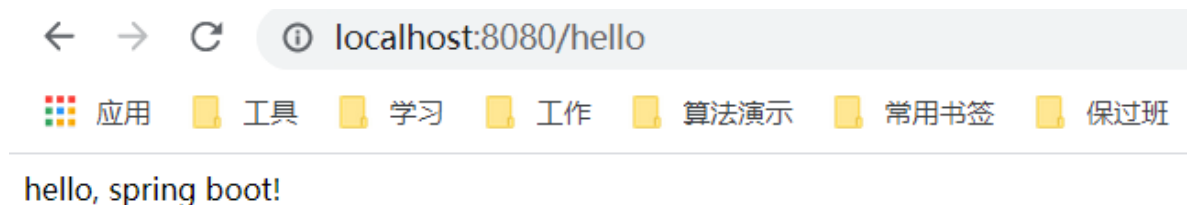
2.3 启动项目

接下来，我们运行main函数，查看控制台：

```
main] c.g.e.s.SpringBootDemo1Application : Starting SpringBootDemo1Application using Java 1.8.0_111 on DESKTOP-J4C0GGK with
main] c.g.e.s.SpringBootDemo1Application : No active profile set, falling back to default profiles: default
main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.50]
main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 925 ms
main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
main] c.g.e.s.SpringBootDemo1Application : Started SpringBootDemo1Application in 2.151 seconds (JVM running for 4.685)
```

- 监听的端口是8080

打开页面访问: <http://localhost:8080/hello>



3 Java配置

在入门案例中, 我们没有任何的配置, 就可以实现一个SpringMVC的项目了, 快速、高效!

但是有同学会有疑问, 如果没有任何的xml, 那么我们如果要配置一个Bean该怎么办? 比如我们要配置一个数据库连接池, 以前会这么玩:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-3.0.xsd"
default-lazy-init="true">
    <!-- 配置连接池 -->
    <bean id="dataSource" class="com.alibaba.druid.pool.DruidDataSource"
        init-method="init" destroy-method="close">
        <property name="url" value="${jdbc.url}" />
        <property name="username" value="${jdbc.username}" />
        <property name="password" value="${jdbc.password}" />
    </bean>
</beans>
```

现在该怎么做呢?

3.1 回顾历史

事实上, 在Spring3.0开始, Spring官方就已经开始推荐使用java配置来代替传统的xml配置了, 我们不妨来回顾一下Spring的历史:

3.1.1 Spring1.x时代

在此时因为jdk1.5刚刚出来, 注解开发并未盛行, 因此一切Spring配置都是xml格式, 想象一下所有的bean都用xml配置, 细思极恐啊, 心疼那个时候的程序员2秒

```
<bean id="helloService" class="com.gpedu.example.service.HelloService" />

<bean id="bookService" class="cn.moondev.service.BookServiceImpl">
    <property name="dependencyService" ref="dependencyService"/>
</bean>
<bean id="otherService" class="cn.moondev.service.OtherServiceImpl">
    <property name="dependencyService" ref="dependencyService"/>
</bean>
<bean id="dependencyService" class="DependencyServiceImpl"/>
```

代码在demo01中

3.1.2 Spring2.x时代

Spring引入了注解开发，但是因为并不完善，因此并未完全替代xml，此时的程序员往往是把xml与注解进行结合，貌似我们之前都是这种方式。

- @Required/@Repository/@Aspect

spring 2.5

- @Component
- @Service
- @Controller
- @RequestMapping

代码在demo02中

```
<context:component-scan base-package="com.gpedu.demo02"/>
```

3.1.3 Spring3.x时代

3.0以后Spring的注解已经非常完善了，因此Spring推荐大家使用完全的java配置来代替以前的xml，不过似乎在国内并未推广盛行。然后当SpringBoot来临，人们才慢慢认识到java配置的优雅。

- @Configuration 去XML化

代码中demo03

- @Import :当我们在web.xml里面配置时，表示指定配置文件，然后回去配置中读取信息，如果我们要导入的别的配置文件，就可以通过import这个配置去做。这个import可以写在applicationContext.xml中，意味着把多个配置文件合并。

代码在demo04中

- @Enable模块驱动

```
@EnableAspectJAutoProxy org.springframework.c...
@EnableLoadTimeWeaving org.springframework.co...
@EnableMBeanExport org.springframework.contex...
@EnableAsync org.springframework.scheduling.a...
@EnableCaching org.springframework.cache.anno...
@EnableScheduling org.springframework.schedul...
@EnableWebMvc org.springframework.web.servlet...

Press Ctrl+空格 to see non-imported classes Next Tip
```

在Spring3.x版本中，集成Redis或者mybatis要怎么做：

1. 导包
2. 创建一个配置类
3. 通过@Bean注解来声明一个bean

mybatis配置举例：

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context-4.3.xsd
        http://www.springframework.org/schema/aop
        http://www.springframework.org/schema/aop/spring-aop-4.3.xsd
        http://www.springframework.org/schema/tx
        http://www.springframework.org/schema/tx/spring-tx-4.3.xsd">

    <!--1 引入属性文件，在配置中占位使用 -->
    <context:property-placeholder location="classpath*:db.properties" />

    <!--2 配置C3P0数据源 -->
    <bean id="datasource" class="com.mchange.v2.c3p0.ComboPooledDataSource"
        destroy-method="close">
        <!--驱动类名 -->
        <property name="driverClass" value="${jdbc.driver}" />
        <!-- url -->
        <property name="jdbcUrl" value="${jdbc.url}" />
        <!-- 用户名 -->
        <property name="user" value="${jdbc.uid}" />
        <!-- 密码 -->
        <property name="password" value="${jdbc.pwd}" />
        <!-- 当连接池中的连接耗尽的时候c3p0一次同时获取的连接数 -->
        <property name="acquireIncrement" value="5"></property>
        <!-- 初始连接池大小 -->
        <property name="initialPoolSize" value="10"></property>
        <!-- 连接池中连接最小个数 -->
        <property name="minPoolSize" value="5"></property>
        <!-- 连接池中连接最大个数 -->
        <property name="maxPoolSize" value="20"></property>
    </bean>

    <!--3 会话工厂bean sqlSessionFactoryBean -->
    <bean id="sqlSessionFactory"
        class="org.mybatis.spring.SqlSessionFactoryBean">
        <!-- 数据源 -->
        <property name="dataSource" ref="datasource"></property>
        <!-- 别名 -->
        <property name="typeAliasesPackage"
            value="com.zhangguo.bookstore.entities"></property>
        <!-- sql映射文件路径 -->
```

```

        <property name="mapperLocations"
value="classpath*:com/zhangguo/bookstore/mapper/*Mapper.xml"></property>
    </bean>

    <!--4 自动扫描对象关系映射 -->
    <bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">
        <!--指定会话工厂，如果当前上下文中只定义了一个则该属性可省去 -->
        <property name="sqlSessionFactoryBeanName" value="sqlSessionFactory">
</property>
        <!-- 指定要自动扫描接口的基础包，实现接口 -->
        <property name="basePackage" value="com.zhangguo.bookstore.mapper">
</property>
    </bean>

    <!--5 声明式事务管理 -->
    <!--定义事物管理器，由spring管理事务 -->
    <bean id="transactionManager"
class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
        <property name="dataSource" ref="datasource"></property>
    </bean>
    <!--支持注解驱动的事务管理，指定事务管理器 -->
    <tx:annotation-driven transaction-manager="transactionManager"/>

    <!--6 容器自动扫描IOC组件 -->
    <context:component-scan base-package="com.zhangguo.bookstore">
</context:component-scan>

    <!--7 aspectj支持自动代理实现AOP功能 -->
    <aop:aspectj-autoproxy proxy-target-class="true"></aop:aspectj-autoproxy>
</beans>

```

通过上面这种方式才能把mybatis注入到我们springIOC容器中，但是这种方式太麻烦了
怎么做更简便呢？

@Enable启动一个模块，把相关组件的bean自动装配到IOC容器中

没有使用Enable启动一个模块时的案例看demo05

使用Enable启动一个模块时的案例看demo06

3.1.4 Spring4.x时代

- @Conditional

代码在demo07中

springboot条件注解

@ConditionalOnBean (仅仅在当前上下文中存在某个对象时，才会实例化一个Bean)

@ConditionalOnClass (某个class位于类路径上，才会实例化一个Bean)

@ConditionalOnExpression (当表达式为true的时候，才会实例化一个Bean)

@ConditionalOnMissingBean (仅仅在当前上下文中不存在某个对象时，才会实例化一个Bean)

@ConditionalOnMissingClass (某个class类路径上不存在的时候，才会实例化一个Bean)

@ConditionalOnNotWebApplication (不是web应用)

3.1.5 IOC流程

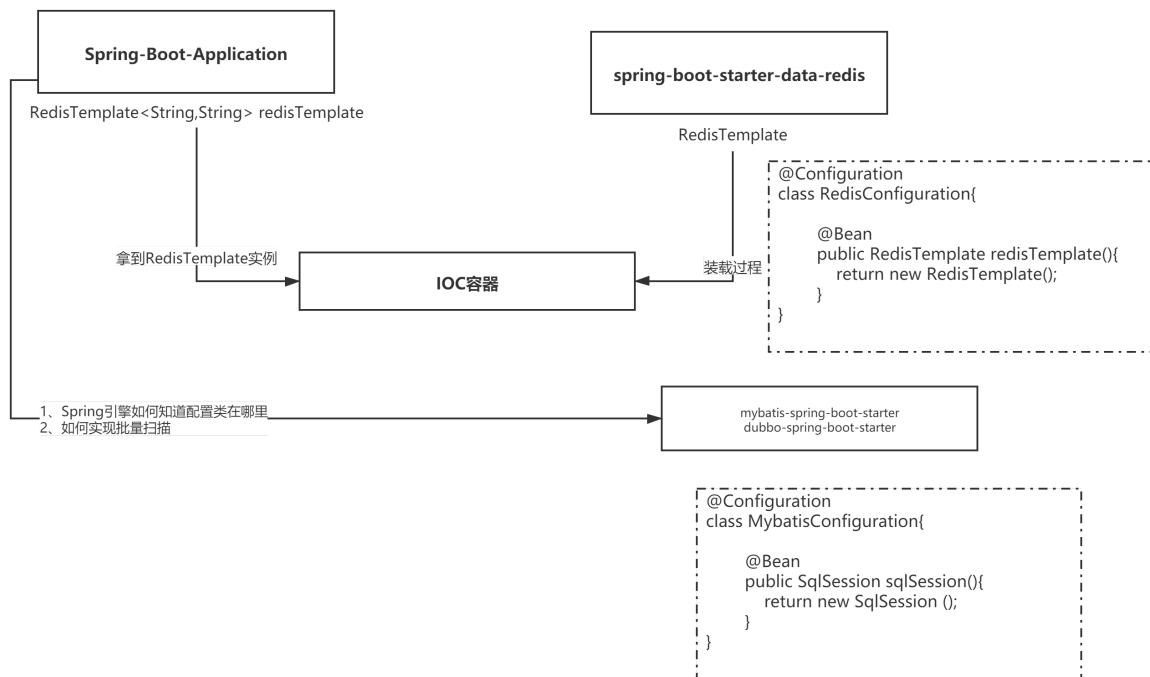
- 1.我们要把托管给spring的bean先扫描到beanDefinition，并初始化Bean容器，放到BeanDefinitionMap容器！
- 2.然后实例化，再进行属性注入（DI）
- 3.初始化Bean,包括是否实现了BeanNameAware等接口
- 4.是否有实现了BeanPostProcessor类，调用postProcessBeforeInitialization
- 5.调用Bean的初始化方法
- 6.是否有实现了BeanPostProcessor类，调用postProcessAfterInitialization，然后这个方法里调用后续aop的逻辑

3.2 springboot启动流程

- 1.初始化
 - 将source放入SpringApplication的source属性中管理，source是一个LinkedHashSet(),这意味着我们可以同时创建多个自定义不重复的Application，但是目前只有一个
 - this.webApplicationType = WebApplicationType.deduceFromClasspath(); 设置web应用类型
 - 从spring.factories中找出ApplicationContextInitializer并设置初始化器initializers
 - 从spring.factories中找出ApplicationListener,并实例化后设置到SpringApplication的监听器listener属性中。这个过程就是找出所有的应用程序时间监听器
 - this.mainApplicationClass = deduceMainApplicationClass(); 找出的main方法的类（这里是StartApplication）,并返回Class对象
- 2.得到监听器，监听run方法
- 3.构造容器环境，包括配置使用的环境文件，并且会通知所有的listener，告知环境准备妥当
- 4.打印banner信息
- 5.创建ApplicationContext会话信息
- 6.准备容器，将信息与会话进行关联并通知
- 7.refreshContext(context) 调用ApplicationContext的refresh()方法，完成IOC

3.3 Springboot 自动装配 (EnableAutoConfiguration)

代码举例见spring-boot-demo中的redis使用



3.3.1 Spring的动态Bean装载

- ImportSelector：代码看demo02、03、04
- Registrar

3.3.2 SpringFactoriesLoader (SPI)

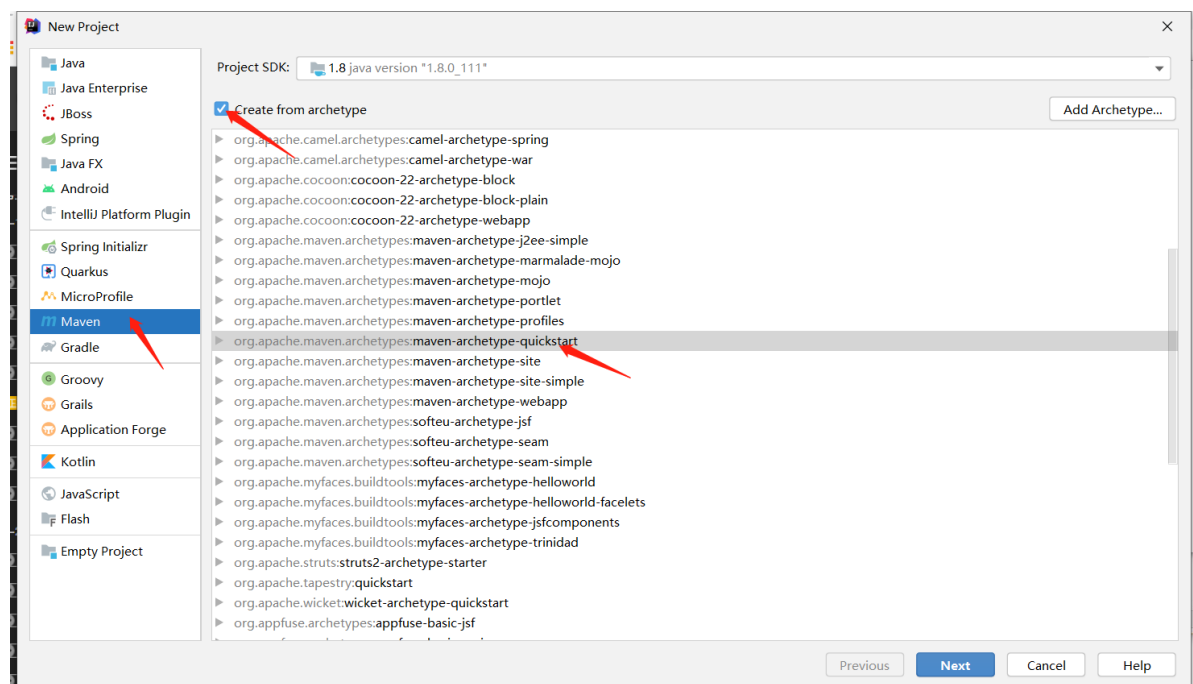
spring.factories（这是springboot里面的一种实现），但是我们JDK里面也提供了一种实现方式就是SPI，我们的Springboot里面的spring.factories都是基于jdk中的SPI思想做的一种实现。

3.3.3 SPI 机制

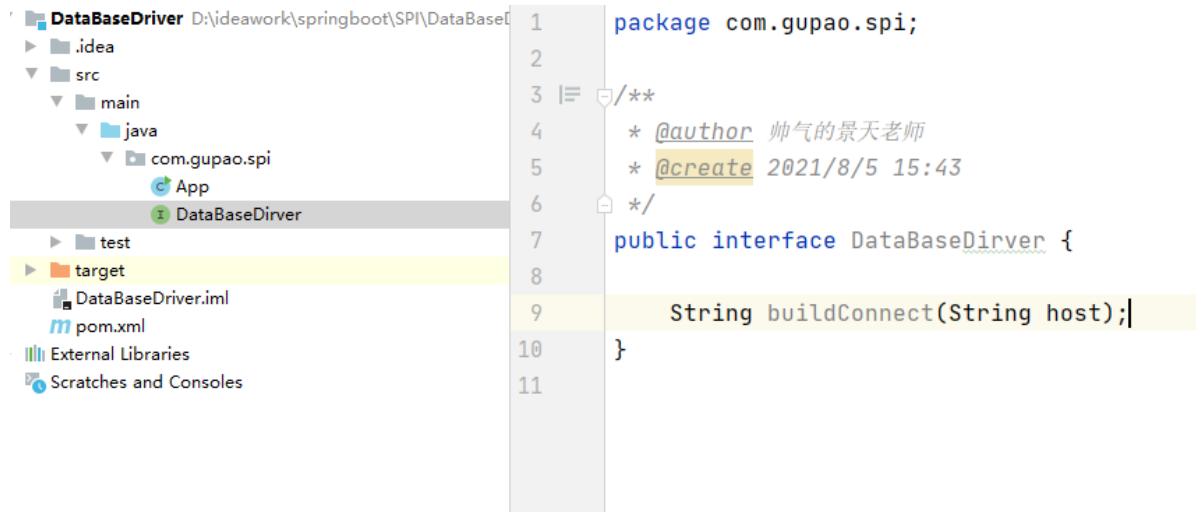
Service provider interface

实现方式：

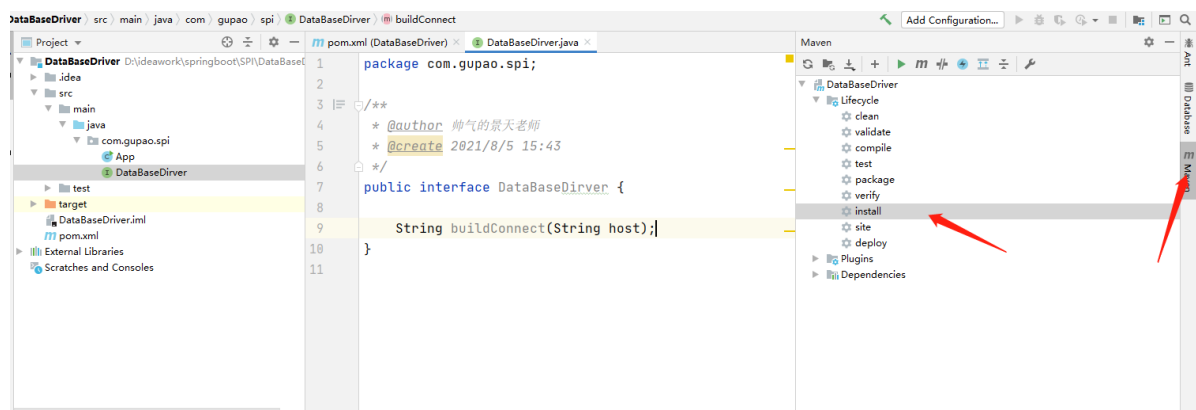
1) 创建接口Maven项目DataBaseDriver



2) 然后创建一个接口DataBaseDriver



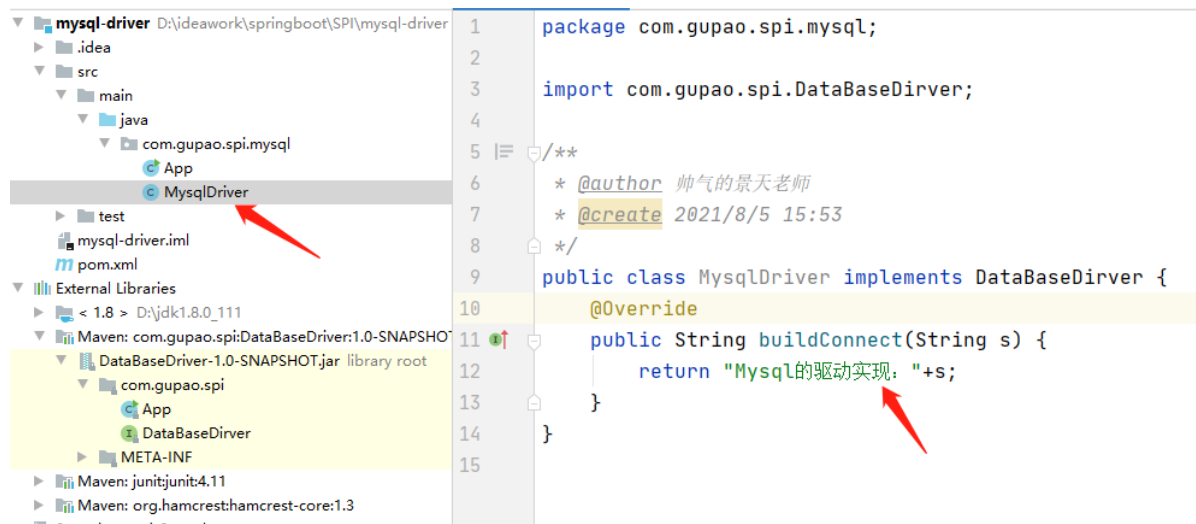
3) 把接口项目打包成jar包



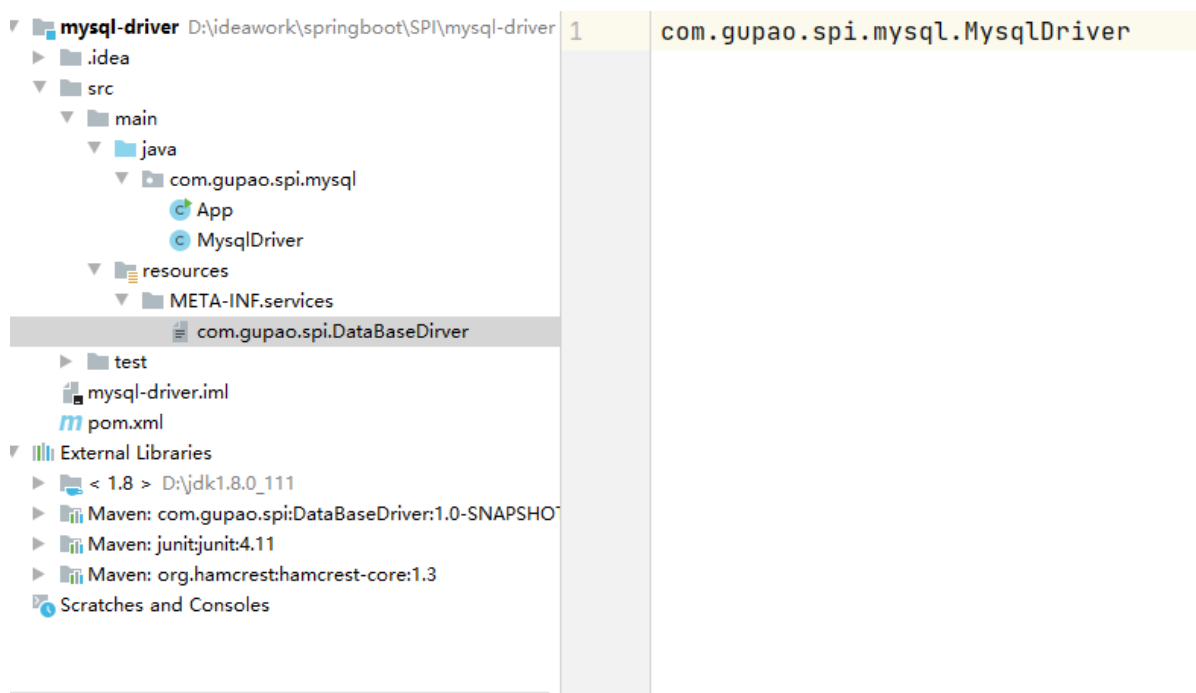
4) 创建mysql-driver的Maven项目，并且引入上面的jar包



5) mysql-driver项目中实现接口



6) 开始扩展，首先在main目录下创建resources目录，然后在resources目录下创建META-INF目录，在META-INF目录下再创建services目录。在services目录下创建com.gupao.spi.DataBaseDriver文件（这个文件名要跟接口的全路径名一致）。



7) 现在就可以开始用它了，我们再创建一个应用程序app-demo

8) 在应用程序中导入接口jar包和实现类jar包

```

</dependency>
<dependency>
    <groupId>com.gupao.spi</groupId>
    <artifactId>DataBaseDriver</artifactId>
    <version>1.0-SNAPSHOT</version>
</dependency>

<dependency>
    <groupId>com.gupao.spi.mysql</groupId>
    <artifactId>mysql-driver</artifactId>
    <version>1.0-SNAPSHOT</version>
</dependency>
</dependencies>

```

9) 写调用方法:



满足以下条件

- 需要在classpath目录下创建一个META-INF/services
- 在该目录下创建一个扩展点的全路径名
 - 文件中填写这个扩展点的实现
 - 文件编码格式UTF-8
- ServiceLoader去进行加载

3.3.4 条件控制

对于官方的starter组件, 例如spring-boot-starter-data-redis,他们的META-INF目录下不存在spring.factories

- 官方包 spring-boot-starter-xxx
 - 官方包的configuration在spring-boot-autoconfigure下, 然后会根据条件进行控制, 如果引入了包就加载, 如果没有引包就不加载



```

8 import ...
9
10 @Configuration(
11     proxyBeanMethods = false
12 )
13 @ConditionalOnClass({RedisOperations.class})
14 @EnableConfigurationProperties({RedisProperties.class})
15 @Import({LettuceConnectionConfiguration.class, JedisConnectionConfiguration.class})
16 public class RedisAutoConfiguration {
17     public RedisAutoConfiguration() {
18     }
19
20     @Bean
21     @ConditionalOnMissingBean(
22         name = {"redisTemplate"}
23     )
24     @ConditionalOnSingleCandidate(RedisConnectionFactory.class)
25     public RedisTemplate<Object, Object> redisTemplate(RedisConnectionFactory

```

- 第三方包 xxx-spring-boot-starter

3.4 Starter组件

手写案例见starter目录下的redisson-spring-boot-starter和spring-boot-demo-02项目

定义：相当于一个模块

读取配置文件：https://blog.csdn.net/qq_37312838/article/details/108237678

3.5 Actuator（监控）

3.5.1 配置

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

```

<http://localhost:8080/actuator/health>

```
{"status": "UP"}
```

<https://docs.spring.io/spring-boot/docs/current/reference/html/actuator.html#actuator.endpoints>

3.5.2 metrics

- JVM（垃圾收集器/内存/堆）
 - <http://localhost:8080/actuator/metrics/jvm.gc.pause>
- 系统（运行时间、平均负载、处理器的信息）
- 线程池信息
- Tomcat会话信息

3.5.3 loggers

<http://localhost:8080/actuator/loggers>

3.5.4 JMX (Java Management Extensions)

3.6 元注解

- @Target 注解可以用在哪。TYPE表示类型，如类、接口、枚举
 - @Target(ElementType.TYPE) //接口、类、枚举
 - @Target(ElementType.FIELD) //字段、枚举的常量
 - @Target(ElementType.METHOD) //方法
 - @Target(ElementType.PARAMETER) //方法参数
 - @Target(ElementType.CONSTRUCTOR) //构造函数
 - @Target(ElementType.LOCAL_VARIABLE) //局部变量
 - @Target(ElementType.ANNOTATION_TYPE) //注解
 - @Target(ElementType.PACKAGE) ///包
- @Retention 注解的保留位置。只有RUNTIME类型可以在运行时通过反射获取其值
 - @Retention(RetentionPolicy.SOURCE) //注解仅存在于源码中，在class字节码文件中不包含
 - @Retention(RetentionPolicy.CLASS) // 默认的保留策略，注解会在class字节码文件中存在，但运行时无法获得，
 - @Retention(RetentionPolicy.RUNTIME) // 注解会在class字节码文件中存在，在运行时可以通过反射获取到
- @Documented 该注解在生成javadoc文档时是否保留
- @Inherited 被注解的元素，是否具有继承性，如子类可以继承父类的注解而不必显式的写下来。